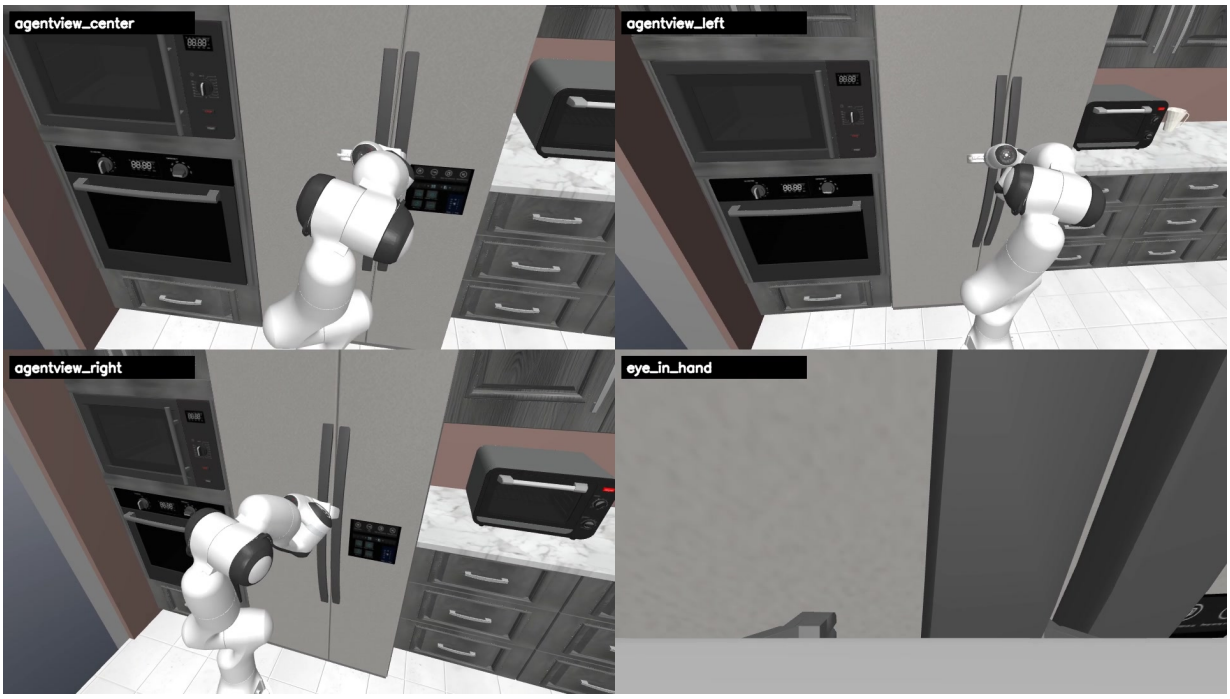
AMD AI DEVMASTER HACKATHON 2026 | TRACK 3

Radeon Physical AI Evidence Suite

Robot learning, simulation, dexterous control, rendering, and safety evidence on AMD Radeon and ROCm.



TEAM	PLATFORMS	DELIVERY
Datawhale-EAI Kewei Chen Yayu Long	Radeon PRO W7900 Ryzen AI MAX+ 395	Source + PDF + 4:59 video JSON + SHA + website

Submission version v1.0.2-amd-hackathon-final | August 2026

Abstract

Radeon Physical AI Evidence Suite is an integrated embodied-AI workflow for AMD Radeon GPUs and ROCm. It connects policy training, simulator execution, closed-loop task evaluation, multi-view rendering, performance telemetry, and SHA-pinned evidence across five complementary settings: Every Embodied VLA manipulation, RoboCasa365 household tasks, DexJoCo dexterous multi-task control, DISCOVERSE simulation and synthetic data, and predictive collision avoidance for Unitree G1. The system runs PyTorch and JAX workloads on AMD Radeon PRO W7900 and AMD Ryzen AI MAX+ 395 hardware. Its central contribution is a unified evidence contract that binds a task protocol, model checkpoint, result JSON, video, runtime manifest, and hash list. The strongest focused result is a SmolVLA checkpoint with 57/60 strict physical successes. At benchmark scale, the suite evaluates GR00T and Pi0.5 on the same RoboCasa365 16-task x 50-episode protocol, reproduces DexJoCo Pi0.5 on native ROCm JAX 0.10, validates DISCOVERSE task and 3DGS paths, and records an 8/8 predictive-CBF Unitree G1 replay panel. The result is a reusable AMD Physical AI engineering stack, a public model and evidence release, and a 4:59 English demonstration film.

Keywords: Physical AI, AMD Radeon, ROCm, VLA, RoboCasa365, DexJoCo, DISCOVERSE, JAX, SmolVLA, Pi0.5, GR00T

Submission at a glance

Item	Frozen submission value
Main learned-policy result	SmolVLA 57/60 strict physical successes
Broadest benchmark	RoboCasa365, 16 tasks x 50 episodes x 2 models
Native AMD frameworks	ROCm PyTorch and ROCm JAX 0.10
Hardware	Radeon PRO W7900 and Ryzen AI MAX+ 395
Demonstration film	4:59, 1080p, English narration and subtitles
Review package	source, PDF, JSON, video, model links, SHA-256, live site

1. Target Application

1.1 Problem

Physical AI projects usually cross several execution layers: simulator physics, camera rendering, data generation, model training, action decoding, closed-loop control, and result presentation. A workflow can appear functional while still failing at a contract boundary such as camera order, action normalization, controller dimensions, or task success semantics. Porting these stacks to a new GPU platform adds framework and kernel compatibility work on top of robotics integration.

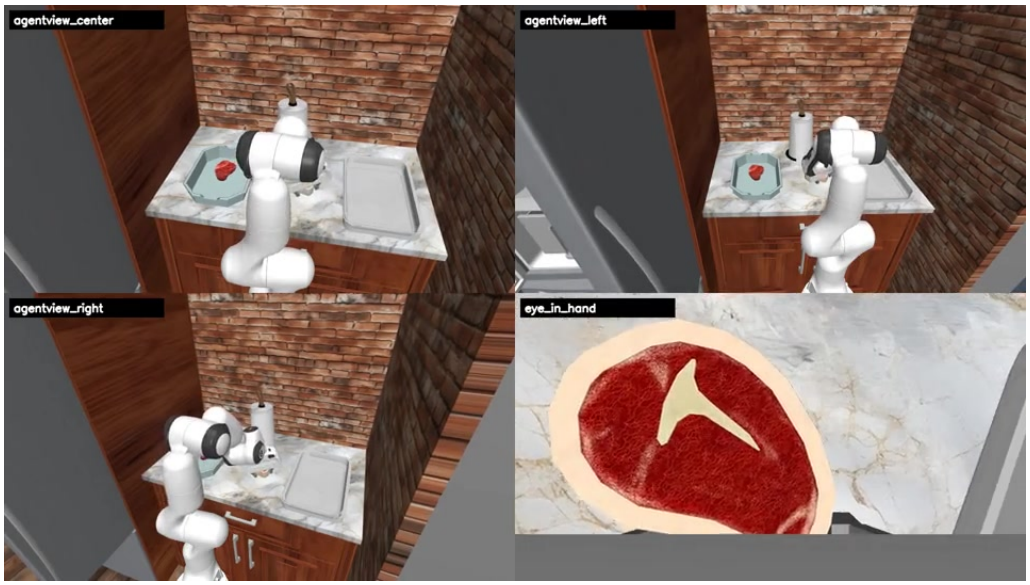
The target application is a reproducible AMD-native laboratory for household and dexterous robot learning. It supports three practical uses:

- train and evaluate robot policies on AMD Radeon GPUs;
- migrate representative simulators and rendering paths to ROCm;
- publish results as linked, machine-checkable evidence rather than isolated

screenshots or unversioned claims.

1.2 Flagship scenario

The flagship scenario evaluates released GR00T and Pi0.5 household policies on the same RoboCasa365 protocol: 16 tasks, 50 closed-loop episodes per task, native task predicates, and synchronized four-view video. The task set covers cabinet, drawer, refrigerator, toaster, sink, dishwasher, sorting, and placement operations.



RoboCasa365 long-horizon meal-packing task

1.3 Design objectives

Objective	Implementation
Broad AMD coverage	PyTorch ROCm, JAX ROCm, MuJoCo/EGL, Genesis AMD, 3DGS
Closed-loop robotics	Simulator state advances from model actions until task termination
Reproducible results	Fixed seeds, explicit denominators, JSON summaries, checkpoint hashes
Reviewable media	Success-first multi-view videos with English narration and subtitles
Reusable engineering	Public setup, training, evaluation, profiling, and validation scripts

2. System Architecture

The suite is organized into six layers. Each layer produces an artifact consumed by the next layer and recorded in the final evidence bundle.

Layer	Main components	Output contract
Tasks and assets	RoboCasa365, DexJoCo, DISCOVERSE, Every Embodied, PAC-MAN	task name, robot, cameras, success predicate
Data	expert trajectories, LeRobot datasets, benchmark JSON	schema, action semantics, statistics, split
Runtime	ROCm PyTorch, ROCm JAX, MuJoCo/EGL, Genesis, 3DGS	device, versions, backend, finite-value gate
Models	SmolVLA, Pi0/Pi0.5, ACT, GR00T, predictive CBF	checkpoint, config, normalization, SHA-256
Evaluation	fixed seeds, task-native predicates, stage metrics	per-episode JSON, aggregate score, video
Publication	GitHub, Hugging Face, GitHub Pages, HyperFrames	release tag, report, demo, evidence index

The W7900 is used for long-running model optimization. The Ryzen AI MAX+ 395 is used for simulator integration, model inference, JAX validation, closed-loop rollouts, rendering, and profiling. This split makes the same project portable across a discrete Radeon GPU and a unified-memory AMD workstation.

2.1 Evidence contract

For every promoted result, the release preserves:

1. task and seed protocol;
2. exact model or controller identity;
3. software and hardware manifest;
4. per-episode and aggregate result JSON;
5. representative video;
6. SHA-256 hashes for the public artifact set.

The included `validate_public_bundle.py` recomputes headline totals from raw JSON and verifies the release file hashes. This provides a fast judge path before a full simulator rerun.

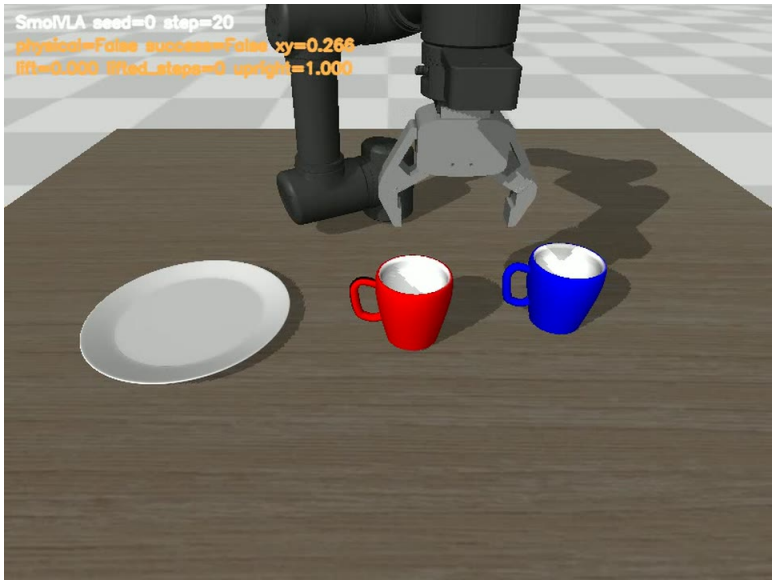
2.2 User-facing workflow

The public site defaults to English and presents successful household, dexterous-hand, simulator, rendering, and humanoid-safety examples first. Detailed benchmark tables, migration notes, reproduction commands, and boundary cases follow. The 4:59 film uses the same archived media and result manifests as the website.

3. Datasets and Task Protocols

3.1 Every Embodied MuJoCo cup manipulation

The SmolVLA workflow uses two language-conditioned variants: place a red mug on a plate and place a blue mug on a plate. Evaluation uses 30 fixed seeds per color. A success requires the physical placement predicate, including lift and stable placement, to pass. The protected weighted continuation checkpoint is evaluated on all 60 episodes.



SmolVLA red mug success

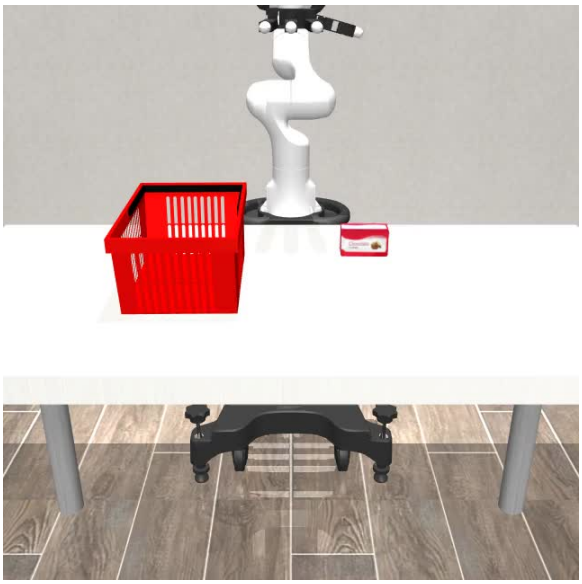
3.2 RoboCasa365

The official fixed-arm evaluation uses 16 tasks with 50 episodes per task for each model. GR00T and Pi0.5 use the same task directories and denominator, which supports direct comparison. The tasks span opening, closing, placement, cleaning, sorting, and appliance interactions.

The official PandaOmron runtime gate additionally validates a 12-dimensional hybrid action contract and three-camera observation path on AMD hardware.

3.3 DexJoCo

DexJoCo contributes 11 dexterous tasks, including bimanual assembly, Hanoi, microwave use, photography, mouse clicking, folding glasses, hammering, bucket pickup, tong pinching, tablet unlocking, and plant watering. The formal panel uses the official multi-task Pi0.5 checkpoint, one episode per task, and seed 0. A separate success-seed archive evaluates fixed seeds 1-10 and stores the first successful rollout available for each additional task.



DexJoCo bucket pickup

3.4 DISCOVERSE and predictive safety

DISCOVERSE coverage includes AIRBOT, MMK2, expert trajectory generation, multi-camera rendering, and 3D Gaussian Splatting. The public HD set contains four official MMK2 state-machine tasks with three native 1920x1080 cameras.

The Unitree G1 safety panel uses eight fixed seeds and predictive perpendicular control-barrier filtering. Each episode records minimum clearance, filtered speed, lateral displacement, threat duration, and escape direction.

4. AMD Radeon and ROCm Implementation

4.1 Hardware

Platform	GPU memory	Workloads in this submission
AMD Radeon PRO W7900	48 GiB	SmolVLA training, ACT training, long policy runs
AMD Ryzen AI MAX+ 395 / Radeon 8060S	unified memory	PyTorch and JAX inference, evaluation, MuJoCo, RoboCasa, DexJoCo, DISCOVERSE, 3DGS

4.2 PyTorch ROCm

The Every Embodied path records PyTorch 2.11.0 with HIP 7.13. The established PyTorch `cuda` API is used by ROCm, so policies select `cuda:0` while the runtime reports an AMD device. Integration work includes uint8 image preprocessing, legacy LeRobot package compatibility, action-chunk timing, headless rendering, and high-frequency telemetry.

4.3 Native JAX ROCm

DexJoCo Pi0.5 runs in an isolated Python 3.12 environment with ROCm 7.14.0, JAX 0.10.0, and JAXlib 0.10.0. The GPU is visible as `rocm:0` on gfx1151. A 64x64 matrix preflight verifies XLA execution before model loading. The official multi-task weights, tokenizer, normalization statistics, environment, and closed-loop policy are then loaded in that environment.

4.4 Simulation and rendering

RoboCasa and RoboSuite use MuJoCo with EGL off-screen rendering. DISCOVERSE uses MuJoCo-based robot environments plus AMD PyTorch model paths. Genesis workloads select the AMD backend where supported. The 3DGS path validates more than one million Gaussian points for each of two renderer targets and exports dynamic Franka and UR5e videos.

4.5 Measured SmolVLA performance on AMD395

Metric	Value
Six-episode physical gate	6/6
Episode elapsed mean / p95	6.547 s / 9.066 s
Aggregate simulator throughput	518.736 sim steps/s
Aggregate control throughput	20.646 action steps/s
Full VLA forward mean / p95	438.332 ms / 440.883 ms
Peak sampled GPU use	61%
Peak sampled VRAM use	11%

5. Model Training and Evaluation

5.1 SmolVLA

The main SmolVLA model uses a 5,000-step parent run followed by a 500-step weighted continuation. The training workflow records loss, optimizer state, normalization statistics, checkpoint metadata, and progress. The selected checkpoint is then loaded into a fresh process for the 60-episode physical gate.

5.2 Pi0/Pi0.5, ACT, and GR00T

Pi0 and ACT are included as policy training and evaluation paths. The protected Pi0 candidate reaches 12/14 on its focused fresh panel, including 9/10 unseen episodes. RoboCasa365 uses the released multi-task GR00T and Pi0.5 checkpoints to measure broad household task performance on an identical 800-episode protocol.

DexJoCo uses the released multi-task Pi0.5 checkpoint for native ROCm JAX closed-loop evaluation. RoboWits adds complete ACT training, checkpointing, inference, and video tooling on W7900 for unexpected-condition manipulation.

5.3 Closed-loop evaluation

A rollout advances the simulator from model actions until the task horizon or success condition. The evaluator saves the seed, task, number of steps, success predicate, video path, and timing. Multi-task totals are computed from the per-task records rather than entered manually.

5.4 Multi-view media

RoboCasa365 exports four-view 1920x1080 household videos. DISCOVERSE exports three native 1080p camera streams and a review composite. DexJoCo publishes one representative success video for each task with an observed success seed. The demo film uses these source clips directly and provides phrase-aligned English narration and subtitles.

6. Results

6.1 Focused VLA result

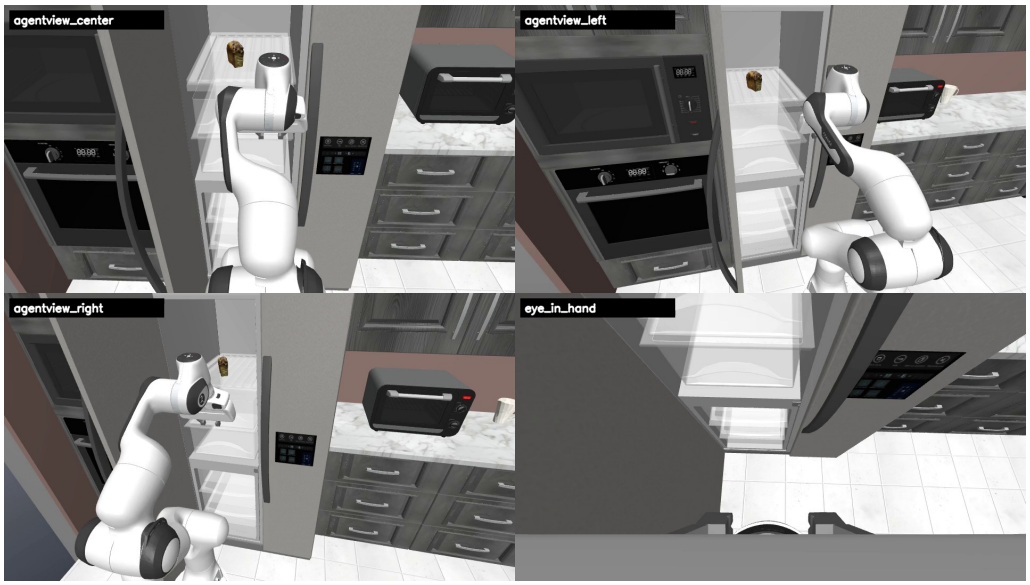
Model	Protocol	Result
SmolVLA weighted500	red/blue physical gate, 30 seeds each	**57/60 (95.0%)**
Pi0 S8500	fresh 14-episode focused panel	**12/14 (85.7%)**

SmolVLA records 27/30 for the red task and 30/30 for the blue task. This is the primary learned-policy result because it combines a released checkpoint, a nontrivial denominator, strict physical success, and representative videos.

6.2 RoboCasa365 broad household benchmark

Model	Tasks	Episodes	Successes	Success rate
GR00T	16	800	**230**	**28.75%**
Pi0.5	16	800	**142**	**17.75%**

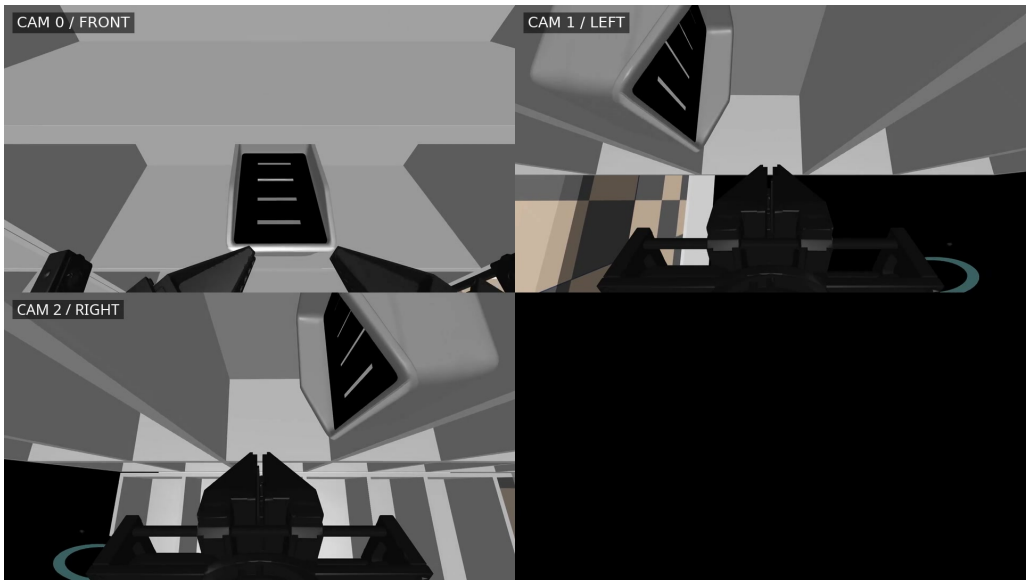
GR00T's strongest tasks include CloseFridge at 39/50, OpenCabinet at 32/50, ScrubCuttingBoard at 26/50, and CloseToasterOvenDoor at 24/50. Pi0.5 records 33/50 on SlideDishwasherRack, 31/50 on PickPlaceDrawerToCounter, 25/50 on OpenDrawer, and 20/50 on OpenCabinet.



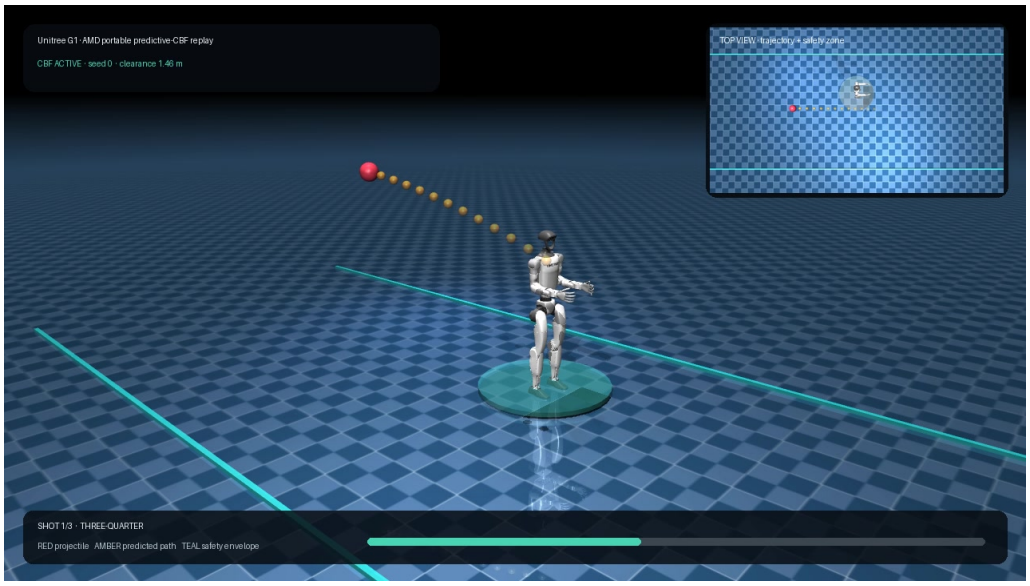
RoboCasa365 four-view household success

6.3 Dexterous and simulator results

Workstream	Protocol	Result
DexJoCo Pi0.5	official multi-task seed 0	**5/11**
DexJoCo success-seed archive	fixed seed search 0-10	success examples for **10/11** tasks
DISCOVERSE HD MMK2	four official state-machine tasks	**4/4**
DISCOVERSE task migration	AIRBOT and MMK2 gates	AIRBOT **12/12**, MMK2 **8/8**
Unitree G1 predictive CBF	fixed seeds 0-7	**8/8**, minimum clearance **0.422 m**



DISCOVERSE three-view box pickup



Unitree G1 predictive CBF replay

7. Innovations and Technical Contributions

7.1 Cross-framework AMD execution

The project demonstrates both ROCm PyTorch and native ROCm JAX in robot policy workloads. This is valuable for Physical AI because current VLA ecosystems are split across the two frameworks. The suite also integrates MuJoCo/EGL, Genesis, and 3DGS into the same AMD-oriented release.

7.2 One evidence model across heterogeneous benchmarks

Each component uses its native task predicate, but publication follows one contract: protocol, checkpoint, manifest, result JSON, media, and SHA. The validator can recompute headline numbers from the submitted JSON. This makes a large multi-project submission reviewable without flattening all tasks into an invalid combined score.

7.3 Household benchmark and mobile runtime integration

The RoboCasa work combines a matched 1,600-episode household benchmark with the official PandaOmron runtime gate. The release validates model loading, native task predicates, four-view video, the 12-D mobile controller, and the three-camera observation path on AMD395.

7.4 Success-first visual evidence

The public site and film bring together DexJoCo dexterous interactions, RoboCasa household workflows, DISCOVERSE multi-view tasks, 3DGS rendering, and Unitree G1 predictive safety. HyperFrames and FFmpeg generate a deterministic 4:59 film with English narration, synchronized subtitles, and stable hashes.

7.5 Open educational value

The release includes public SmolVLA, Pi0, ACT, and RoboWits model repositories, native training notebooks, AMD setup notes, profiling scripts, evaluation scripts, and an English engineering website. These materials let learners run a trained policy first, inspect a successful video, then repeat training and evaluation on AMD hardware.

8. Reproducibility and Deliverables

8.1 Public deliverables

Deliverable	Location
Dedicated source repository	https://github.com/Ethan-Chen-plus/radeon-physical-ai-evidence-suite
Frozen source release	v1.0.2-amd-hackathon-final
Reproducibility guide	`docs/REPRODUCIBILITY.md`
Technical report	`output/pdf/datawhale-eai-radeon-physical-ai-technical-report.pdf`
4:59 English demo	https://ethan-chen-plus.github.io/amd-physical-ai-showcase/assets/videos/amd-physical-ai-demo-en.mp4
Interactive showcase	https://ethan-chen-plus.github.io/amd-physical-ai-showcase/
Result evidence	`evidence/` and website JSON downloads
Model artifacts	Datawhale Hugging Face repositories linked from the README

8.2 Judge quick path

```
git clone https://github.com/Ethan-Chen-plus/radeon-physical-ai-evidence-suite.git
cd radeon-physical-ai-evidence-suite
git checkout v1.0.2-amd-hackathon-final
python3 scripts/validate_public_bundle.py
```

The command validates public result totals and hashes without downloading large models. Full environment and simulator commands are in `docs/REPRODUCIBILITY.md`.

8.3 Release integrity

The source tag, PDF, evidence JSON, documentation, and validator are listed in the repository `SHA256SUMS`. Video hashes and language variants are stored in `evidence/demo-release.json`. Model pages publish their own file metadata and checksums.

8.4 Dependency and license handling

Large upstream assets, datasets, and checkpoints are downloaded from their official or public hosts. Assets with restricted redistribution terms remain on their official hosts. `THIRD_PARTY_NOTICES.md` identifies the major upstream projects; original integration code is released under Apache 2.0.

9. Completed Scope

The frozen release contains five fully indexed workstreams: focused VLA policy training, matched RoboCasa365 household evaluation, native ROCm JAX dexterous control, DISCOVERSE simulation and rendering, and Unitree G1 predictive safety control. Each promoted result is connected to its protocol, checkpoint identity, result record, representative media, runtime manifest, and integrity hash.

10. Team and Contributions

Kewei Chen - project lead and primary maintainer

- designed the cross-project system and release architecture;
- migrated and validated PyTorch, JAX, simulation, rendering, and policy paths

on AMD Radeon hardware;

- implemented data, training, inference, evaluation, telemetry, video, and SHA

workflows;

- produced the technical report, demo film, reproducibility package, and public

website;

- maintained the Datawhale-EAI submission and upstream attribution.

Yayu Long - evaluation and learning experience contributor

- validated learner-facing AMD environment setup and notebook workflows;
- reviewed evaluation records, representative videos, and task-to-evidence

links against the documented protocols;

- contributed to reproduction documentation and final submission quality

assurance.

Datawhale community contributors and all upstream project authors are credited through their repositories and licenses. Their work provides the model, simulator, dataset, and benchmark foundations on which this AMD integration is built.

11. Conclusion

Radeon Physical AI Evidence Suite demonstrates that a diverse modern robotics stack can execute on AMD Radeon and ROCm across PyTorch, JAX, simulation, rendering, policy learning, and closed-loop evaluation. The project contributes more than a single demo: it supplies a reusable engineering and evidence system, public model and result artifacts, broad household and dexterous benchmarks, a PandaOmron mobile runtime gate, and a polished review interface. The fixed release makes every headline result traceable to code, protocol, JSON, video, and hash.

References

1 Datawhale Every Embodied. https://github.com/datawhalechina/every-embodied

2 Hugging Face LeRobot. https://github.com/huggingface/lerobot

3 RoboCasa. https://github.com/robocasa/robocasa

4 DISCOVERSE. https://github.com/discoverse-dev/DISCOVERSE

5 DexJoCo. https://github.com/brave-eai/dexjoco

6 RoboWits. https://umass-embodied-agi.github.io/RoboWits/

7 PAC-MAN predictive CBF implementation. https://github.com/lzyang2000/perceptive_cbf_rl

8 AMD ROCm JAX documentation. https://rocm.docs.amd.com/projects/ai-ecosystem/en/latest/frameworks/jax/install.html